Cluster-Decomposed Beam Search for Rubik's Cubes and Squares up to $N = 20$

An Open Engine, Verified Commutator Actions and a Size-Generalizing Heuristic

Gaurav
Shiv Nadar University
Greater Noida, India

Shree
Shiv Nadar University
Greater Noida, India

Sunaina
Shiv Nadar University
Greater Noida, India

Kashvi
Shiv Nadar University
Greater Noida, India

Shaurya
Shiv Nadar University
Greater Noida, India

Sharanya
Shiv Nadar University
Greater Noida, India

ABSTRACT

Learned Rubik's Cube solvers stop at the 5×5×5, and they train a separate network for every cube size. Classical solvers handle any size but say little about how learned search scales. We study the full family of N×N×1 Rubik's Squares and N×N×N Rubik's Cubes for every N from 3 to 20 on one open engine. The engine generates moves from geometry, splits any puzzle into Demaine's cubie clusters with a sticker order that does not depend on N, checks solvability, and re-verifies every reported solution. On it we build a staged beam search (Method 5B) whose stages each solve a few pieces. Its heuristics are exact abstraction tables and small cluster networks, and its actions include commutators that are machine-verified to touch only the pieces they should. On a two-core CPU without a GPU, Method 5B solved all 140 random Squares we tested (3×3×1 to 20×20×1) and all 16 random cubes (3×3×3 to 20×20×20); the 20×20×20 took 79 minutes. Its solutions are long, however: on Squares small enough for exhaustive search, a per-size CayleyPy-style network (Method 4) found the optimum on every instance, while Method 5B averaged 1.2 and 7.0 moves above it on the 3×3×1 and 4×4×1, and its length divided by $N^2/\ln N$ keeps rising with N. A cluster-token transformer trained only on Squares with $N \le 8$ (Method 5A) keeps a correlation of 0.53 with the true scramble depth at $N = 20$. We release the engine, benchmarks, solvers and all result files, and report our negative results alongside the positive ones.

CCS CONCEPTS

• **Computing methodologies** → **Search methodologies**; *Machine learning*; • **Theory of computation** → Mathematical optimization.

KEYWORDS

Rubik's Cube, heuristic search, beam search, pattern databases, macro-operators, commutators, size generalization, Cayley graphs

1 INTRODUCTION

The N×N×N Rubik's Cube is a standard testbed for planning on graphs that are far too large to enumerate. Its worst-case solution length, God's number, is 20 for $N = 3$ [19] and grows as $\Theta(N^2/\log N)$ [9]; finding a shortest solution of a given state is NP-complete [10]. Learned solvers have recently become strong on small cubes. DeepCubeA [1] and EfficientCube [20] solve the 3×3×3, and the CayleyPy approach of Chervov et al. [7] reaches near-optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers. All of these train one network per size on a one-hot encoding of the whole sticker array, so a 6×6×6 needs a new model and a new training run, and none reports a cube beyond $N = 5$.

We ask what it takes to cover the whole range $3 \le N \le 20$ with search-based solvers, for both the cube and its flat relative, the N×N×1 Rubik's Square of Demaine et al. [9]. The two families share their key structure. Moves never mix *cubie clusters*, and clusters of the same type behave identically at every N. That structure is what we build on. Our contributions:

(1) **An open engine and benchmark** (§4) for both puzzle families and every N. Moves come from 3D geometry rather than hand-written tables; clusters are found automatically and given a sticker order that does not depend on N, so equal cluster types have equal *signatures*. A solvability checker, three move metrics and a harness that re-applies every returned solution make results comparable across methods.
(2) **Method 5B, a staged beam search** (§5) that solves a few pieces per stage. It combines exact abstraction tables (pattern databases [8]), small cluster networks, exact breadth-first search for stages whose projected puzzle is tiny, and commutator macro-actions [15] that are generated and then machine-verified to move only the intended pieces. It solved every random state we tested, from the 3×3×1 to the 20×20×1 and from the 3×3×3 to the 20×20×20, on a CPU.
(3) **Method 5A, a cluster-token transformer** (§6) that reads the puzzle as a set of cluster tokens, so one network applies to any N. Trained only on Squares with $N \le 8$, its distance estimates stay correlated with scramble depth up to $N = 20$.
(4) **A careful evaluation** (§8) on shared, seeded instances, including exact optima for small Squares, a reproduction of the CayleyPy recipe (Method 4), ablations, and counting lower bounds. We report where our methods fall short: Method 5B's solutions are long, and we could not reproduce CayleyPy's 3×3×3 result without a GPU.

2 RELATED WORK

Structure and complexity. Demaine et al. [9] decompose N×N×1 and N×N×N puzzles into cubie clusters, show that every cluster can be solved without disturbing the others, and obtain $\Theta(N^2/\log N)$

bounds on the diameter by solving many identical clusters with one shared move sequence. Optimal solving is NP-complete for both families [10]. For the 3×3×3 the diameter is exactly 20 in the half-turn metric [19].

Classical solvers. Korf found optimal 3×3×3 solutions with IDA* and pattern databases [16], following Culberson and Schaeffer's pattern databases [8]. Kociemba's two-phase algorithm [14] and its engineered implementation min2phase [5] give near-optimal 3×3×3 solutions in milliseconds. Large cubes are usually solved by *reduction* to a 3×3×3 [23]. Korf's macro-operators [15] learn move sequences that change one part of the state and restore the rest; our verified commutators play the same role inside a learned search. The Kaggle Santa 2023 competition [12] included large cubes; the KMCoders write-up [13] reports, for example, a 33×33×33 in 4,719 moves using cluster-wise 3-cycles.

Learned solvers. DeepCubeA [1] learns a cost-to-go function by approximate value iteration and searches with batch weighted A*. EfficientCube [20] trains directly on random scrambles from the goal. Q* search [2] scores all children of a node in one network pass and handles large action spaces, including 1,872 meta-actions on the 3×3×3. CayleyPy [7] trains a residual MLP to predict the length of a random walk from the solved state (a diffusion distance) and searches with very wide beams; it reports over 98% optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers, and its authors note that the diameter is unknown for every $N > 3$ [6]. All of these models are trained per puzzle size.

Other uses of the cube. The cube also serves as a benchmark for robotic manipulation [18] and for spatial reasoning in (multimodal) language models [3, 4]; those works use small cubes and do not target solution quality at scale.

Size generalization. Networks that read a problem as a set or sequence of parts, such as pointer networks [22] and set transformers [17], can be applied to larger instances than they were trained on. Method 5A applies this idea to cubie clusters.

3 PROBLEM

Puzzles. An N×N×1 Rubik's Square is an $N \times N$ grid of cubies with a top and a bottom colour, plus side stickers on the border. Its $2N$ moves are 180° flips of one row or one column; each move is its own inverse. An N×N×N Rubik's Cube has $6N^2$ stickers and $3N$ slices; counting a quarter, half or three-quarter turn of one slice as one move (the half-turn metric, HTM) gives $9N$ moves. A state is *solved* when every face shows one colour, in any of the reachable orientations (24 for the cube, 4 for the Square). Given a reachable state, the task is to return a short move sequence that solves it.

Metrics. We report lengths after merging commuting same-axis moves (Section 4) in HTM, the quarter-turn metric (QTM, where a half turn counts 2) and the slice-turn metric (STM, where a block of adjacent layers turned together counts 1). For the Square all moves are flips, so HTM = QTM. To compare sizes we also divide lengths by $N^2/\ln N$, the growth rate of the worst case.

Clusters. A cubie can only ever reach the positions in its own cluster. Table 1 lists the cluster types and how many of each an N has; our engine finds them automatically (§4) and the counts are checked by unit tests. A 20×20×20 has 2,400 stickers, 9 wing clusters and 81 center clusters; every one must be solved.

Table 1: Cubie cluster types. Wings combine two sticker orbits of one layer. Counts are for one N; $k = \lfloor N/2 \rfloor - 1$.

Puzzle	Cluster type	Stickers	Count
Cube	corners	24	1
	middle edges	24	1 if N odd
	wing edges	48	k
	centers (+, ×, oblique)	24	$((N-2)^2 - [N \text{ odd}])/4$
	fixed centers	6	1 if N odd
Square	corner	16	1
	edge	12	$2k$
	interior	8	k^2
	edge-cross, cross, center	6, 4, 2	odd N only

4 ENGINE AND BENCHMARK

All methods run on one engine [11]. Everything below is generic over cuboid dimensions (d_0, d_1, d_2), so the cube (N, N, N) and the Square $(N, N, 1)$ come from the same code.

Moves from geometry. Each sticker stores a doubled, centred cubie coordinate and an outward normal. A move rotates every sticker of one slab by an integer rotation matrix; looking up the rotated (position, normal) pair gives a permutation π, and applying the move is the gather $s \leftarrow s[\pi]$. A slab whose cross-section is not square, such as one row of the Square, only gets 180° turns, and an axis of length 1 gets none. Unit tests check that every move has order 4 (or 2) and that each move followed by its inverse is the identity, for every tested N.

Canonical form. Moves on the same axis commute. We canonicalise a sequence by merging each same-axis run per layer and sorting by layer, and we use the same order to prune search: after a move on axis a, layer ℓ, no move on axis a with layer $\leq \ell$ is allowed. This removes immediate repeats and every redundant ordering of commuting moves.

Clusters with N-invariant signatures. Union-find over all move permutations gives the sticker orbits; orbits sharing cubies are merged into cubie clusters. Inside a cluster, stickers are sorted by face and by the signs and relative magnitudes of their in-face coordinates. This order does not depend on N, so the moves acting on a cluster induce the same local permutations at every N. We hash the local generator set and the solved pattern into a *signature*; tests confirm that, for example, every ×-center cluster of the 5×5×5, 7×7×7 and 9×9×9 has the same signature. A model trained for one cluster type therefore applies at any size.

Projections. Restricting attention to a subset of stickers gives a *projected puzzle*. When it is small we enumerate it by breadth-first search from the solved pattern; the resulting table gives exact distances for that subset and doubles as an exact membership test.

Solvability checker. A state is accepted only if it passes: global and per-cluster colour counts; membership in the exact projection table

for every cluster small enough to enumerate (all Square clusters); for cubes, valid corner, middle-edge and wing pieces, corner twist $\equiv 0 \pmod 3$ and, for odd N after normalising orientation by the fixed centers, even middle-edge flip and equal corner and middle-edge permutation parity; and for the Square, joint consistency of each cross cluster with the border clusters it shares moves with. These are necessary conditions. Property tests confirm that the checker accepts scrambled states and rejects a single corner twist, a single middle-edge flip, a swap of two corners on odd cubes and wrong colour counts.

Harness. Every solver implements one interface and returns engine moves. The harness re-applies the moves, decides *solved* itself, and writes one row per instance with lengths in all three metrics, time, nodes, memory, seed, hardware and git commit. A crash or time-out counts as a failure and stays in the denominator.

Shared beam search. Methods 4, 5A and 5B share one beam search. Per step it expands the beam in chunks, scores children with the heuristic, and keeps the W best after removing duplicates by a random 64-bit linear hash (also across the last three steps). Only (score, hash, parent, action) is stored per child; the next beam is rebuilt from parents. Beam states are packed at 3 bits per sticker and can live on the device, in pinned host memory or on disk. The search accepts *macro actions*: fixed move sequences applied as one step and expanded into ordinary moves in the output. A heuristic that can score all children of a parent in one pass is used that way, as in Q* search [2].

Instances. Random states are long random canonical walks (at least 25× the number of moves, and at least 400 moves). This approximates, but does not guarantee, a uniform random reachable state. Every instance is regenerated from a seed and checked by the solvability checker; all methods see the same instances.

5 METHOD 5B: CLUSTER-DECOMPOSED STAGED BEAM SEARCH

Method 5B solves the puzzle in a sequence of stages (Figure 1, Algorithm 1). Each stage adds a few pieces to an exact goal: those pieces, and every piece solved by an earlier stage, must show their solved stickers. Each stage is a small search problem with a sharp signal, and later stages cannot silently undo earlier work.

Stage plan. For the Square we follow Demaine et al. [9]: the *frame* (corner, center and edge-cross clusters), then edge clusters two at a time, then cross clusters and interior clusters three at a time, from the outside in. For the cube we use an *edges-first* order: fixed centers, corners, middle edges, wings, then centers, with every wing and center cluster split into stages of 6 pieces. Inner-slice moves never disturb corners, and the commutators below never disturb edges, so the later and more numerous stages have cheap actions that keep earlier work.

Stage actions. A move that touches none of the current targets cannot make progress, so each stage searches only the single moves that touch a target plus the macro actions that do. This cuts the branching factor from $O(N^2)$ macros to a handful. The macro pool is:

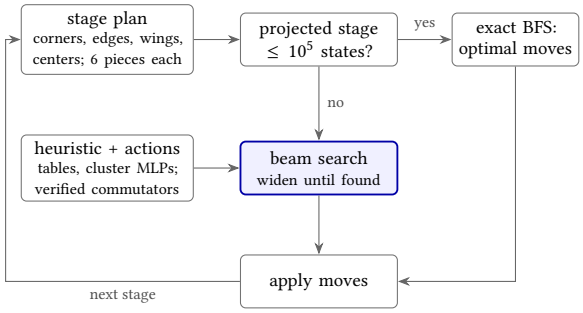


Figure 1: Method 5B. Each stage adds a few pieces to an exact goal. Tiny stages are solved optimally by lookup; the rest by beam search over single moves and verified commutators, guided by exact abstraction tables and small cluster networks.

- *Square*: for every row r and column c, the commutator $c\,r\,c\,r$, which changes only the cluster at their crossing (Demaine's Lemma 2), and the conjugates $r\,c\,r$ and $c\,r\,c$; $3N^2$ macros.
- *Cube*: $X\,Y\,X'\,Y'$ for an inner-slice quarter turn X and an outer-face quarter turn Y on another axis, which never moves corners; and pure-center commutators $[A, B\,C\,B'] = A\,B\,C\,B'\,A'\,B\,C'\,B'$ with A and C inner-slice quarter turns on one axis (the middle slice included, e.g. $[M, U\,r\,U']$) and B a face turn on another axis. Every candidate is applied to the sticker permutation and kept only if it moves center stickers alone; duplicates with the same permutation are dropped. This leaves 192 pure-center commutators for the 4×4×4 and 29,376 for the 20×20×20, and covers all 81 center clusters of the 20×20×20.

Once every corner and edge is solved, a center stage searches *only* the pure-center commutators, which keep all earlier work by construction.

Heuristics. A stage's heuristic is a weighted sum over clusters:

$$h(s) = \sum_{c \in T} h_c(s) + \lambda_{\text{keep}} \sum_{c \in P} h_c(s) + \lambda_{\text{ex}} \left|\{i \in G : s_i \neq s^*_i\}\right|, \qquad (1)$$

where T are the target clusters, P the earlier clusters that some allowed action can move, G the stage's goal stickers and s^* the solved state ($\lambda_{\text{keep}} = 1$, $\lambda_{\text{ex}} = 0.5$). Each per-cluster term h_c, shared by signature across all N, is one of three kinds:

(1) *Exact table*, when the projected cluster has at most 1.5M states: every Square cluster and the fixed centers.
(2) *Coarse abstraction tables*: the projection after mapping one pair of opposite colours to 1 and the rest to 0. These are pattern databases over a colour abstraction [8] and give admissible lower bounds. They have 2,187 states for corners, 126,720 for middle edges and 735,471 for every center type; wings are skipped because every such projection exceeds the cap. For corners a finer three-class abstraction (one colour to 1, its opposite to 2) has 153,090 states.
(3) *Cluster MLP*: a small residual MLP (hidden width 256, two residual blocks) trained on random walks of the projected cluster to predict walk length, combined with the coarse

Algorithm 1: Method 5B (one state)

Input: state s, stage plan $S_1, \ldots, S_m$
Output: move sequence μ with $s \cdot \mu$ solved

$\mu \leftarrow [\,]$;
for $i \leftarrow 1$ **to** m **do**
  $G \leftarrow \bigcup_{j \le i} \text{goal}(S_j)$;
  if *s already solved on G* **then continue**;
  $A \leftarrow$ single moves and verified macros that move some target of S_i;
  if *projection of G under A has $\le 10^5$ states* **then**
    $v \leftarrow$ optimal moves by greedy descent on its BFS table;
  else
    $h \leftarrow$ stage heuristic (Eq. 1);
    $W \leftarrow W_0$;
    repeat
      $v \leftarrow \textsc{BeamSearch}(s, A, h, W, \text{goal} = G)$;
      $W \leftarrow 4W$;
    until *v found* ***or*** $W > W_{\max}(|A|)$;
  if *no v* **then return** failure;
  $s \leftarrow s \cdot v$; $\mu \leftarrow \mu \,\|\, v$;
return canonicalise(μ);

tables by max. Six such networks (corner, middle edge, wing, three center types) cover every cube size; each trained for 1,500 steps on CPU in 2–15 minutes.

Corners use the three-class tables alone. The MLP's walk-length targets overestimate the true distance, and taking the max with them flattens the tables' signal; Section 8.5 measures the effect.

Exact small stages. If the projection of a stage's goal stickers under the stage's single moves has at most 100,000 states, we enumerate it and descend greedily on the exact distances, which is optimal for that stage. The Square frame always qualifies: for any N it depends only on rows and columns 0, $\lfloor N/2 \rfloor$ and $N-1$, and is therefore a 3×3×1 with at most 768 states.

Width schedule. Each stage starts at width W_0 (128 for the Square, 512 for the cube) and, if it fails within its step limit (40 and 60 steps), retries at 4× the width up to $W_{\max}$ (2,048 and 8,192). Stages with few actions may widen further while $W \times |\text{actions}| \le 1.2 \cdot 10^6$; the 18-action 3×3×3 corner stage thus reaches 2^{16}.

6 METHOD 5A: ONE NETWORK FOR EVERY SIZE

Method 5A replaces the per-size sticker network with one that reads the puzzle as a set of cluster tokens. Each cluster becomes one token: the one-hot colours of its stickers in the canonical order, projected by a linear map shared by all clusters of the same type, plus a learned embedding of the cluster kind and a small MLP of geometric features (the cluster's two in-face depths divided by $N-1$, and $N/20$). A transformer encoder [21] without positional order mixes the tokens, since they form a set [17]. The value head is *additive*: each token predicts a non-negative cost, the costs are summed, and a small global correction is added. A sum matches the structure of the problem, since total distance grows with the number of unsolved clusters, and it extends to any number of tokens. The per-token costs also serve as per-cluster heuristics inside Method 5B's stages.

Training uses fresh non-backtracking random walks from the solved state on every training size, with the walk length k as target and the loss on $(\hat{k}-k)/(N^2/\ln N)$ so that all sizes contribute on one scale. The model we evaluate has 198,658 parameters (width 64, two layers, four heads) and was trained for 2,500 steps on Squares with $N \in \{4, \ldots, 8\}$ only, in 35 minutes on CPU. At test time it runs full-puzzle beam search for $N \le 7$ and, above that, Method 5B's stages with its per-token costs as the stage heuristic. The architecture also has a Q*-style move head; we did not evaluate it.

7 METHOD 4: CAYLEYPY REPRODUCTION

Method 4 re-implements the recipe of Chervov et al. [7] on our engine: one residual MLP per puzzle size on one-hot sticker inputs, trained with MSE on fresh non-backtracking random walks to predict walk length, and beam search with hash deduplication for at most 200 steps. Square models have 0.25–0.45M parameters and were trained on 4M (N ≤ 5) or 10M (N = 6–8) examples in 1–4 minutes each on CPU. We also trained the paper's 4.0M-parameter 3×3×3 model on 32M examples (77 minutes on CPU), 250× fewer than the paper's 8B.

8 EXPERIMENTS

Setup. All runs used a 2-core CPU, no GPU and one thread per process. Square experiments use 10 seeded random states for $N \le 12$ and 5 for $13 \le N \le 20$; cube experiments use 2 states for $N \le 10$ and one each for $N = 15$ and $N = 20$. Every method sees the same instances, and every number below comes from the harness's result files, from which our tables and figures are generated by script. Means are over solved instances; a superscript gives the solved fraction when it is below 100%.

8.1 Coverage: every size, both puzzles

Method 5B solved all 140 of 140 Square instances (Table 2) and all 16 of 16 cube instances (Table 3). The 20×20×1 took 60 s per state on average; the 15×15×15 took 31 minutes (5,280 moves) and the 20×20×20 79 minutes (9,904 moves). To our knowledge no earlier learned-search solver has been reported beyond the 5×5×5 [7]. The price is length: Method 5B's 3×3×3 solutions average 52 HTM, against an optimum of about 18 on random states [19].

8.2 Distance from optimal on small Squares

The 3×3×1 and 4×4×1 are small enough to enumerate. Breadth-first search from all four solved orientations visits 768 and 82,944 states; their diameters are 6 and 10 moves. Table 4 scores each method against the exact optimum on the same instances. The per-size network of Method 4 found the optimum on every instance (10/10 and 10/10). Method 5B was 1.2 and 7.0 moves above it on average: its stages fix pieces in a prescribed order and pay for commutators that a global search does not need.

8.3 Comparison with per-size networks

Method 4's per-size networks are the strongest method we have on the sizes where they exist. On the 60 Square instances with $3 \le N \le 8$ that both methods solved, Method 5B's solutions were on average

Table 2: Rubik's Square, random states. Columns: number of instances; mean optimal length (exact BFS, $N \le 4$); mean HTM of Method 4 (per-size network, W=4096), Method 5A and Method 5B on the same instances; Method 5B's solved fraction, its length divided by $N^2/\ln N$, and its mean time per state. A superscript gives the solved fraction when below 100%; "–" = not run (no model for that size), and "$-^{0/k}$" = tried, none solved.

N	n	optimal	Method 4	Method 5A	Method 5B HTM ± 95% CI	Method 5B solved	Method 5B/ $(N^2/\ln N)$	Method 5B time (s)
3	10	3.6	3.6	–	4.8 ± 1.0	10/10	0.59	0.0
4	10	6.4	6.4	6.4	13.4 ± 1.8	10/10	1.16	0.0
5	10	–	8.8	10.0	22.2 ± 2.3	10/10	1.43	0.2
6	10	–	13.0	15.0	28.0 ± 2.5	10/10	1.39	0.1
7	10	–	15.0	$23.5^{4/10}$	41.8 ± 4.4	10/10	1.66	0.2
8	10	–	20.4	56.4	61.6 ± 5.4	10/10	2.00	0.6
9	10	–	–	70.0	83.2 ± 6.1	10/10	2.26	1.0
10	10	–	–	$85.5^{8/10}$	103.4 ± 6.1	10/10	2.38	1.6
11	10	–	–	–	121.4 ± 7.5	10/10	2.41	3.3
12	10	–	–	$-^{0/1}$	143.0 ± 6.4	10/10	2.47	5.9
13	5	–	–	–	170.4 ± 7.3	5/5	2.59	7.0
14	5	–	–	–	221.2 ± 17.6	5/5	2.98	11.9
15	5	–	–	–	242.8 ± 20.9	5/5	2.92	13.0
16	5	–	–	–	280.0 ± 15.8	5/5	3.03	15.8
17	5	–	–	–	298.4 ± 30.8	5/5	2.93	21.1
18	5	–	–	–	354.8 ± 11.5	5/5	3.17	34.0
19	5	–	–	–	394.4 ± 23.5	5/5	3.22	43.0
20	5	–	–	–	486.4 ± 43.3	5/5	3.64	60.3

Table 3: Rubik's Cube, random states, Method 5B. Time is wall-clock seconds per state.

N	n	solved	HTM	QTM	HTM/$(N^2/\ln N)$	time (s)
3	2	2/2	52.0 ± 15.7	54.5	6.3	19
4	2	2/2	246.0 ± 27.4	249.5	21.3	26
5	2	2/2	452.5 ± 2.9	459.0	29.1	62
6	2	2/2	724.5 ± 110.7	729.5	36.1	109
7	2	2/2	1,045.0 ± 33.3	1,049.5	41.5	161
8	2	2/2	1,510.5 ± 120.5	1,515.0	49.1	269
10	2	2/2	2,260.5 ± 22.5	2,270.5	52.0	530
15	1	1/1	5,280.0	5,292.0	63.5	1,875
20	1	1/1	9,904.0	9,916.0	74.2	4,760

Table 4: Optimality on the smallest Squares. "Gap" = mean HTM above the exact optimum over solved instances; "optimal" = instances solved in exactly the optimal length. Method 5A was evaluated for $N \ge 4$ only.

Puzzle	God's	mean opt.	Method	solved	gap	optimal
3×3	6	3.6	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	–	–	–
			Method 5B	10/10	1.2	1/10
4×4	10	6.4	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	10/10	0.0	10/10
			Method 5B	10/10	7.0	0/10

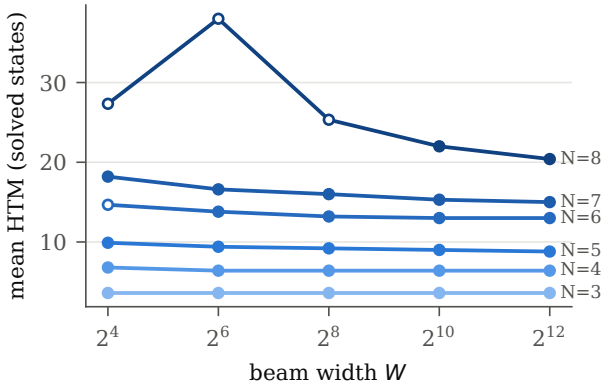


Figure 2: Method 4: mean solution length against beam width W, one per-size network for each Square size $N = 3, \ldots, 8$ (10 states each; means over solved states). Hollow markers: not all 10 states solved, so the mean covers only the easier ones.

2.35× as long as Method 4's at W=4096 (Table 2). As CayleyPy reports for cubes [7], wider beams help (Figure 2). They also decide coverage at the largest trained size: on the 8×8×1, Method 4 solved 3/10 states at W=16, 9/10 at W=256 and all from W=1024 on, at 17.4 s per state for W=4096. The cost is a separate model and training run for every size; we have them only for $N \le 8$.

8.4 Does one network generalise across sizes?

Table 5 shows that the distance estimate of Method 5A does not collapse on sizes it never saw: its correlation with the true walk

Table 5: Method 5A, trained on Squares with $N \in \{4, \ldots, 8\}$ only: agreement of its distance estimate with the true random-walk depth at the end of training, on fresh walks (* = size never seen in training). RMSE is in units of $N^2/\ln N$.

N	4	5	6	7	8	10*	14*	20*
Pearson r	0.48	0.62	0.67	0.61	0.64	0.66	0.65	0.53
normalised RMSE	0.71	0.63	0.58	0.61	0.57	0.57	0.60	0.70

depth is 0.64 at $N = 8$, the largest training size, 0.66 at $N = 10$ and 0.53 at $N = 20$. These correlations are moderate; a model of 198,658 parameters trained for 2,500 CPU steps is far from converged.

As a solver, Method 5A solved these fractions of instances: N=4: 10/10, N=5: 10/10, N=6: 10/10, N=7: 4/10, N=8: 10/10, N=9: 10/10, N=10: 8/10, N=12: 0/1. With full-puzzle beam search it matched or came close to Method 4's per-size networks for $N \leq 6$, including the optimum on all ten 4×4×1 instances (Table 4), but at $N = 7$ six of ten full-puzzle searches used up their step limit without reaching the goal: the search horizon, not time, was the problem. Inside Method 5B's stages ($N \geq 8$) it solved most states (the two 10×10×1 failures hit the 300 s budget), with solutions shorter than Method 5B's own: on the 62 instances both solved, Method 5B's solutions were 1.65× as long as Method 5A's overall and 1.20× as long on the 18 instances with $N > 8$, sizes outside Method 5A's training range. The cost is time: transformer scoring on CPU made Method 5A's staged solves several times slower than Method 5B's (Table 2). The one 12×12×1 we tried did not finish within 900 s, and we stopped the planned $N = 12$ and $N = 14$ runs there; larger sizes need a GPU or a cheaper scoring path such as the move head.

8.5 Ablations

Table 6 removes one component at a time.

Macro actions are what make large Squares fast. Without them the 8×8×1 still solves at the same length but takes 15 s instead of 0.9 s per state, and at $N = 12$ only 4/5 states solve within 600 s (111 s per state against 4.5 s). A commutator turns a "break, then restore" detour, which the summed heuristic scores as uphill, into one downhill step.

Exact small stages shorten solutions rather than speed them up: solving the frame optimally instead of by beam search gives 59 instead of 67 moves at $N = 8$ and 144 instead of 175 at $N = 12$.

On the cube, *restricting late center stages to pure-center commutators* left lengths unchanged here and saved time (74 s against 133 s per 5×5×5 state). The *corner heuristic* result is mixed: taking max(MLP, tables) instead of the tables alone made 4×4×4 solutions longer (296 against 246 moves) and 5×5×5 solutions shorter (421 against 452), and was slower on both. Removing the coarse tables, so that corners fall back to the MLP alone, gave the same pattern (296 and 436 moves). With two states per size we read these cube differences as indicative only; the clearest cube effect is speed.

8.6 Scaling with N and lower bounds

Figure 3 divides each mean length by $N^2/\ln N$. For Method 5B on the Square this ratio rises from 0.59 at $N = 3$ to 3.64 at $N = 20$, and on the cube it keeps rising across the whole range. Over the sizes we can test, Method 5B's lengths therefore grow faster than the $N^2/\log N$ rate that Demaine's construction achieves. The likely cause is that Method 5B solves clusters a few at a time, while that construction amortises one move sequence over many identical clusters. The per-size network of Method 4 sits well below Method 5B where it exists.

Table 7 puts the counting bounds next to our lengths. The bounds only say that *some* state needs that many moves, so they cannot certify optimality of any single solution, but on the cube our lengths are more than an order of magnitude above them, so either the bounds are very weak or there is much room left; both are likely.

9 LIMITATIONS

- *Small samples and CPU only.* We test few states per size, especially for large cubes (one each at $N = 15$ and 20). The coverage results are strong evidence of reliability, not a guarantee, and every timing is for a 2-core CPU.
- *Long solutions.* Method 5B is far from optimal, and its normalised length rises over the tested range of N. It solves clusters a few at a time and never batches identical fixes across clusters the way Demaine's construction does, which is what $O(N^2/\log N)$ requires.
- *Method 4 on cubes.* With 250× less training data than the original, our 3×3×3 model solved short scrambles but none of 10 random states at $W \in \{256, 1024, 4096\}$ and none of 4 at $W = 16{,}384$ (from the project's status log). This is a statement about our compute, not about the method.
- *Method 5A's evidence is preliminary.* One small Square model, a CPU budget, and no cube model.
- *Checker and sampling.* The checker enforces necessary conditions only, and random states come from long random walks rather than an exact uniform sampler.
- *Missing baselines.* Our project plan also includes IDA* with pattern databases, a reduction solver with Kociemba's two-phase algorithm, and Demaine's parallel algorithm; they are not yet implemented, so we compare only against Method 4 and exact optima.

10 CONCLUSION

Decomposing Rubik's puzzles into clusters with N-invariant signatures lets one engine, one set of small heuristics and one search procedure cover every size from 3 to 20. With exact abstraction tables, machine-verified commutator actions and exact small stages, a staged beam search solved every random state we tested on a CPU, including a 20×20×20. The same structure gives a cluster-token network whose distance estimates carry over to Square sizes it never saw. What remains is solution quality: per-size networks are much shorter where they apply, and closing that gap at large N, for example with Method 5A's heuristics inside the stages and Demaine-style batching of identical fixes, is the next step. Code, data and result files are available at https://github.com/gauravrai1704/Rubik.

ACKNOWLEDGMENTS

This work was carried out as a course project at Shiv Nadar University.

Table 6: Ablations of Method 5B, removing one component at a time: macro actions, exact small stages, the restriction of late center stages to pure-center commutators, the tables-only corner heuristic, and the coarse abstraction tables (corners then use the MLP alone). Each cell: solved / tried, mean HTM over solved states, mean seconds per state (failures included in time). Square: 5 states per size, 600 s budget; cube: 2 states per size, 300 s budget. The same instances as Tables 2 and 3.

Square	$N = 8$			$N = 12$		
	solved	HTM	s	solved	HTM	s
full Method 5B	5/5	59	0.9	5/5	144	4.5
− macros	5/5	59	15	4/5	136	111
− exact stages	5/5	67	0.5	5/5	175	4.0

Cube	$N = 4$			$N = 5$		
	solved	HTM	s	solved	HTM	s
full Method 5B	2/2	246	31	2/2	452	74
− center restriction	2/2	246	41	2/2	452	133
corners: max(MLP, tab.)	2/2	296	89	2/2	421	125
− coarse tables	2/2	296	87	2/2	436	120

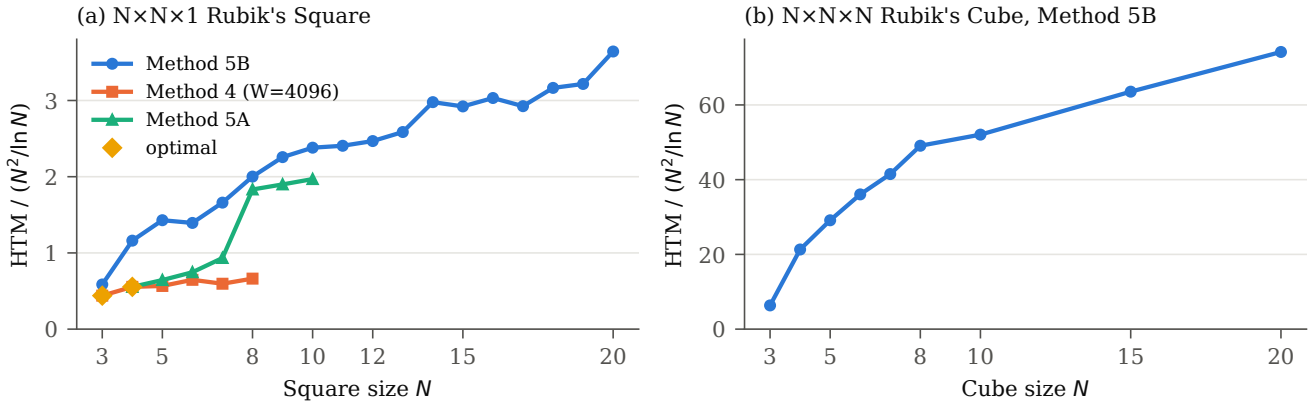


Figure 3: Solution length divided by $N^2/\ln N$, the growth rate of the worst case, for the Square (a) and the cube (b). A flat line would mean growth at the rate of the worst case. Means over solved states; the same instances as Tables 2 and 3.

Table 7: Counting lower bounds on the worst case (God's number) in our move sets ($2N$ Square flips, $9N$ cube half-turn moves), from the number of reachable center-cluster configurations as in Demaine et al. [9], next to Method 5B's mean length on random states. The bounds are for the worst state, not for these instances.

	Square		Cube	
N	lower bound	Method 5B mean	lower bound	Method 5B mean
6	2	28	35	724
8	5	62	75	1,510
10	9	103	126	2,260
15	18	243	262	5,280
20	39	486	557	9,904

REFERENCES

[1] Forest Agostinelli, Stephen McAleer, Alexander Shmakov, and Pierre Baldi. 2019. Solving the Rubik's Cube with Deep Reinforcement Learning and Search. *Nature Machine Intelligence* 1, 8 (2019), 356–363. https://doi.org/10.1038/s42256-019-0070-z

[2] Forest Agostinelli, Shahaf S. Shperberg, Alexander Shmakov, Stephen McAleer, Roy Fox, and Pierre Baldi. 2024. A* Search Without Expansions: Learning Heuristic Functions with Deep Q-Networks. arXiv:2102.04518 [cs.AI]

[3] Dhruv Anand and Ehsan Shareghi. 2025. Cube Bench: A Benchmark for Spatial Visual Reasoning in MLLMs. arXiv:2512.20595

[4] Huan ang Gao, Zikang Zhang, Tianwei Luo, Kaisen Yang, Xinzhe Juan, Jiahao Qiu, Tianxing Chen, Bingxiang He, Hao Zhao, Hao Zhou, Shilong Liu, and Mengdi Wang. 2025. CubeBench: Diagnosing Interactive, Long-Horizon Spatial Reasoning Under Partial Observations. arXiv:2512.23328

[5] Shuang Chen. [n. d.]. min2phase: An Implementation of Kociemba's Two-Phase Algorithm. https://github.com/cs0x7f/min2phase. Accessed October 2026.

[6] Alexander Chervov, Dmytro Fedoriaka, Mark Obozov, Elena Konstantinova, et al. 2025. CayleyPy Growth: Efficient Growth Computations and Hundreds of New Conjectures on Cayley Graphs. arXiv:2509.19162

[7] Alexander Chervov, Kirill Khoruzhii, Nikita Bukhal, Jalal Naghiyev, Vladislav Zamkovoy, Ivan Koltsov, Lyudmila Cheldieva, Arsenii Sychev, Arsenii Lenin, Mark Obozov, Egor Urvanov, and Alexey Romanov. 2025. A Machine Learning Approach That Beats Rubik's Cubes. In *Advances in Neural Information Processing Systems (NeurIPS)*. Spotlight. arXiv:2502.13266.

[8] Joseph C. Culberson and Jonathan Schaeffer. 1998. Pattern Databases. *Computational Intelligence* 14, 3 (1998), 318–334.

[9] Erik D. Demaine, Martin L. Demaine, Sarah Eisenstat, Anna Lubiw, and Andrew Winslow. 2011. Algorithms for Solving Rubik's Cubes. In *Proceedings of the 19th Annual European Symposium on Algorithms (ESA 2011) (Lecture Notes in Computer Science, Vol. 6942)*. Springer, 689–700. Full version: arXiv:1106.5736.

[10] Erik D. Demaine, Sarah Eisenstat, and Mikhail Rudoy. 2018. Solving the Rubik's Cube Optimally is NP-complete. In *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 96)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 24:1–24:13. https://doi.org/10.4230/LIPIcs.STACS.2018.24

[11] Gaurav, Shree, Sunaina, Kashvi, Shaurya, and Sharanya. 2026. Rubik: Six Solvers for N×N×1 and N×N×N Rubik's Puzzles on One Engine. https://github.com/gauravrai1704/Rubik.

[12] Kaggle. 2023. Santa 2023 – The Polytope Permutation Puzzle. https://www.kaggle.com/competitions/santa-2023.

[13] KMCoders. 2024. Kaggle Santa 2023 – The Polytope Permutation Puzzle: KMCoders' Solution. https://github.com/wata-orz/santa2023_permutation_puzzle.

[14] Herbert Kociemba. [n. d.]. The Two-Phase Algorithm. https://kociemba.org/cube.htm. Accessed October 2026.
[15] Richard E. Korf. 1985. Macro-Operators: A Weak Method for Learning. *Artificial Intelligence* 26, 1 (1985), 35–77.
[16] Richard E. Korf. 1997. Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. 700–705.
[17] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
[18] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. 2019. Solving Rubik's Cube with a Robot Hand. arXiv:1910.07113
[19] Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge. 2014. The Diameter of the Rubik's Cube Group Is Twenty. *SIAM Rev.* 56, 4 (2014), 645–670. https://doi.org/10.1137/14097375X
[20] Kyo Takano. 2023. Self-Supervision Is All You Need for Solving Rubik's Cube. *Transactions on Machine Learning Research* (2023). arXiv:2106.03157.
[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[22] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[23] Daniel Walton. [n. d.]. rubiks-cube-NxNxN-solver. https://github.com/dwalton76/rubiks-cube-NxNxN-solver. Accessed October 2026.